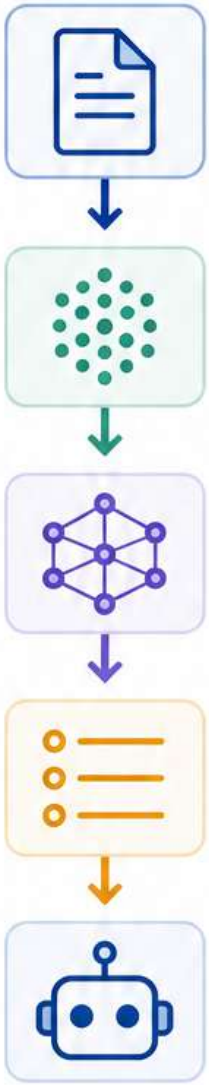
01

How Vector Indexes Work

The Hidden Engine Behind Fast Semantic Search



Learn how AI retrieves the most relevant information in milliseconds.

VANSH BUDHIRAJA

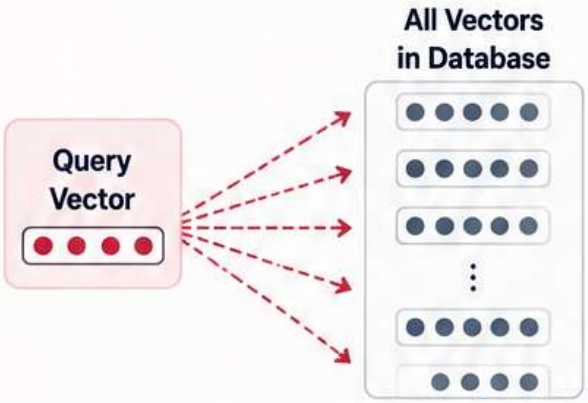
Why Do We Need a Vector Index?

Finding similar items in high-dimensional space is hard.
A vector index helps us do it in **milliseconds**.

WITHOUT INDEX

Brute Force Search

Compare with every vector



Time Complexity: O(N × D)
Must compare with all N vectors of dimension D

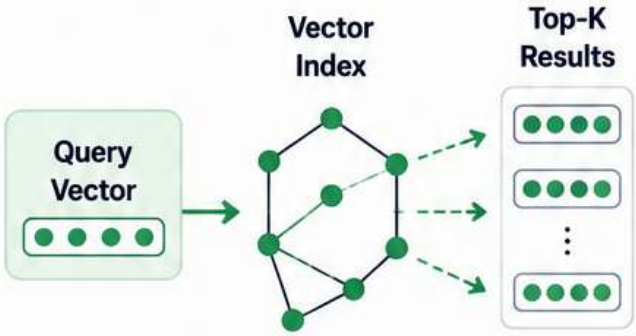
Very Slow
Gets extremely expensive as data grows

Doesn't Scale
Not practical for millions or billions of vectors

WITH INDEX

Indexed Search

Search only likely candidates



Very Fast
Search only a small set of promising vectors

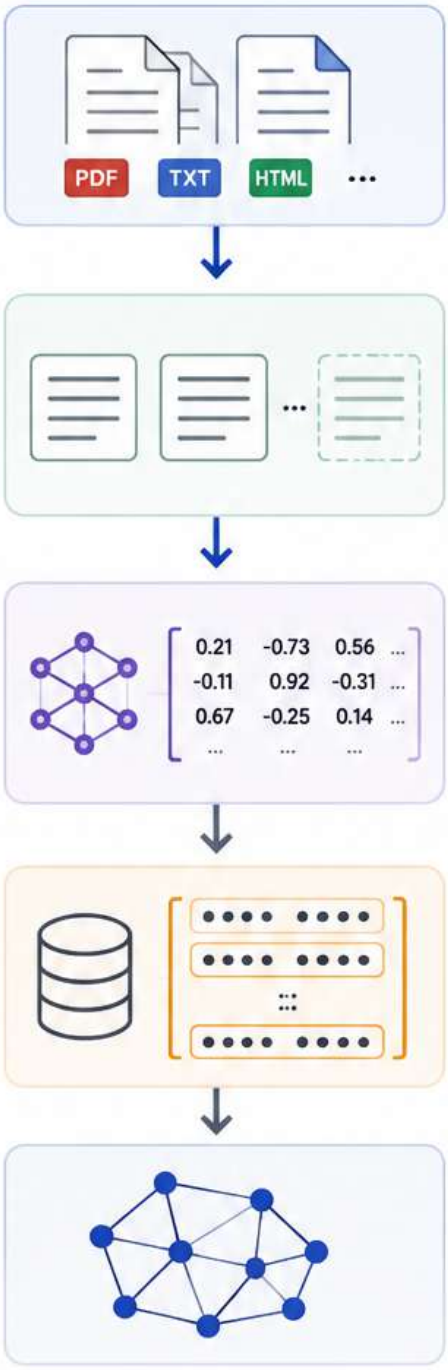
Scales Effortlessly
Handles millions or billions of vectors with low latency

High Efficiency
Returns similar results with excellent accuracy

Vector indexes turn an expensive brute force search into an **efficient** search that **scales to real-world data.**

Building the Index

Before we can search, we need to convert our data into **vectors** and organize them in an **index**.



1 Collect Documents

Gather data from multiple sources such as PDFs, websites, databases, APIs, and more.

2 Chunk the Documents

Split documents into smaller, meaningful chunks so each piece can be represented separately.

3 Generate Embeddings

Convert each chunk into a high-dimensional vector using an embedding model.

4 Store the Vectors

Store the embeddings in a vector database for efficient retrieval.

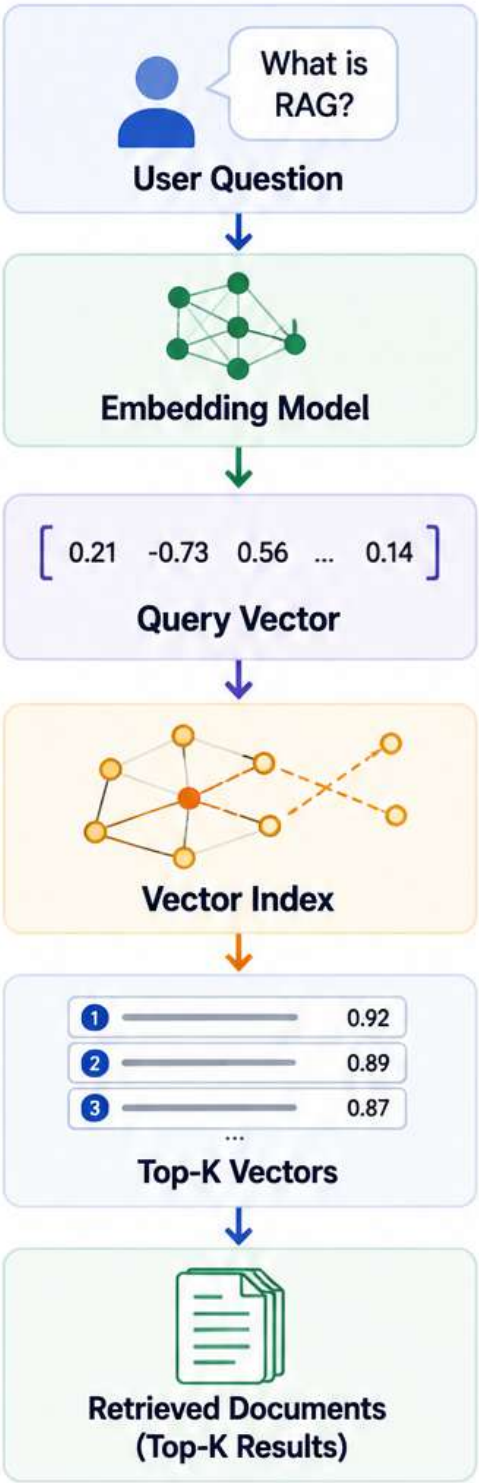
5 Build the Index

Organize the vectors into an index structure optimized for fast similarity search.

The index is the key that makes similarity search **fast, scalable**, and **production-ready**.

How Search Works

When a user asks a question, the system converts it into a **vector**, searches the **index**, and returns the most **similar results**.



1 User Question

The user asks a question in natural language. This is the start of the retrieval process.

2 Embedding Model

The question is converted into a high-dimensional vector using an embedding model.

3 Query Vector

The output is a dense vector that captures the semantic meaning of the question.

4 Vector Index

The query vector is used to search the index. The index quickly finds the most similar vectors without scanning the entire dataset.

5 Top-K Vectors

The index returns the Top-K most similar vectors along with their similarity scores.

6 Retrieved Documents

These vectors are mapped back to the original document chunks and passed to the LLM for answer generation.

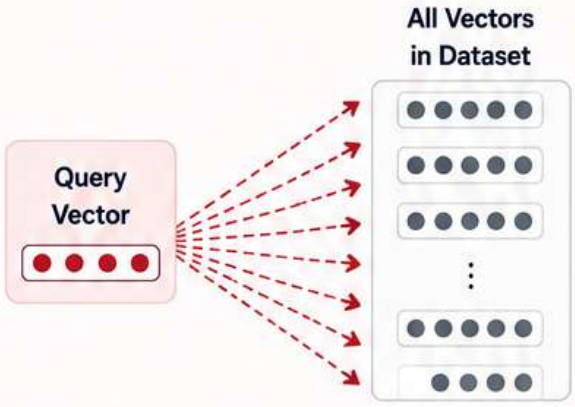
The vector index acts like a smart search engine for meaning.
It finds the right context—fast.

Exact Search vs Approximate (ANN) Search

Vector indexes use **Approximate Nearest Neighbor (ANN)** algorithms to deliver lightning-fast results with high accuracy.

EXACT SEARCH (BRUTE FORCE)

Compare the query vector with **every** vector in the dataset.



Highest Accuracy
Always returns the true nearest neighbors.

Very Slow
Time complexity: O(N × D)
(N = number of vectors, D = dimension)

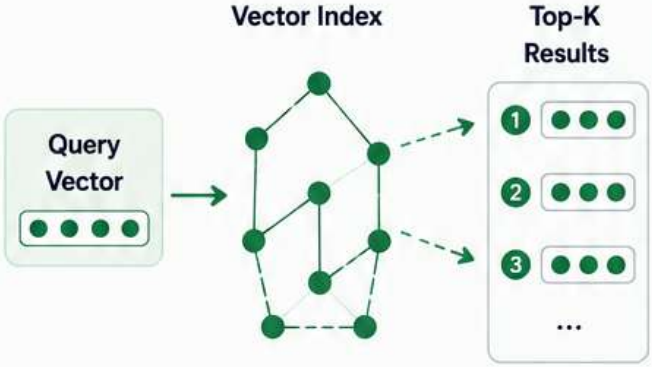
Doesn't Scale
Becomes extremely expensive as data grows.

Best For
Small datasets where speed is not a major concern.

VS

APPROXIMATE SEARCH (ANN)

Search a **small set of promising candidates** instead of the entire dataset.



Very Fast
Sub-linear time complexity
(e.g., O(log N) or close to it).

High Accuracy
Returns results very close to the true nearest neighbors.

Scales to Billions
Handles massive datasets with low latency.

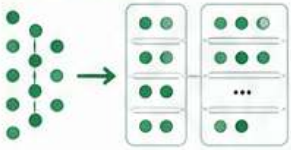
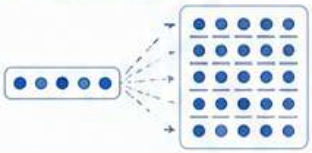
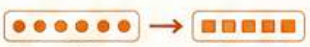
Best For
Large-scale systems where speed and scalability are critical.

Key Takeaway

ANN trades a tiny bit of accuracy for massive gains in speed and scalability.
That's how AI systems search billions of vectors in milliseconds.

Popular Index Types

Different index structures optimize the speed–accuracy trade-off in different ways.

INDEX TYPE	HOW IT WORKS	BEST FOR	SPEED	ACCURACY	MEMORY
HNSW (Hierarchical Navigable Small World)	Builds a multi-layer graph. Search starts from the top layer and navigates down to find nearest neighbors. 	High recall requirements and low latency needs. Most general-purpose use.	**Very Fast** (Sub-linear)	**High** (Excellent Recall)	**Medium** (More memory overhead)
IVF (Inverted File)	Vectors are clustered using k-means. During search, only the closest clusters are scanned. 	Very large datasets where speed is critical. Good balance of speed and accuracy.	**Fast** (Sub-linear)	**Medium to High** (Depends on # of clusters probed)	**Low to Medium** (Efficient for very large datasets)
Flat (Exact Search)	Brute-force search. Compare the query with every vector in the dataset. 	Small datasets or use cases where 100% accuracy is mandatory.	**Slow** (Linear)	**Highest** (100% Exact)	**High** (Stores all vectors)
PQ (Product Quantization)	Compress vectors into compact codes using quantization. Search is done in the compressed space.  Original Vector → Compressed Code	Memory-constrained environments with massive datasets.	**Fast** (Sub-linear)	**Medium** (Some loss due to compression)	**Very Low** (Highly memory efficient)

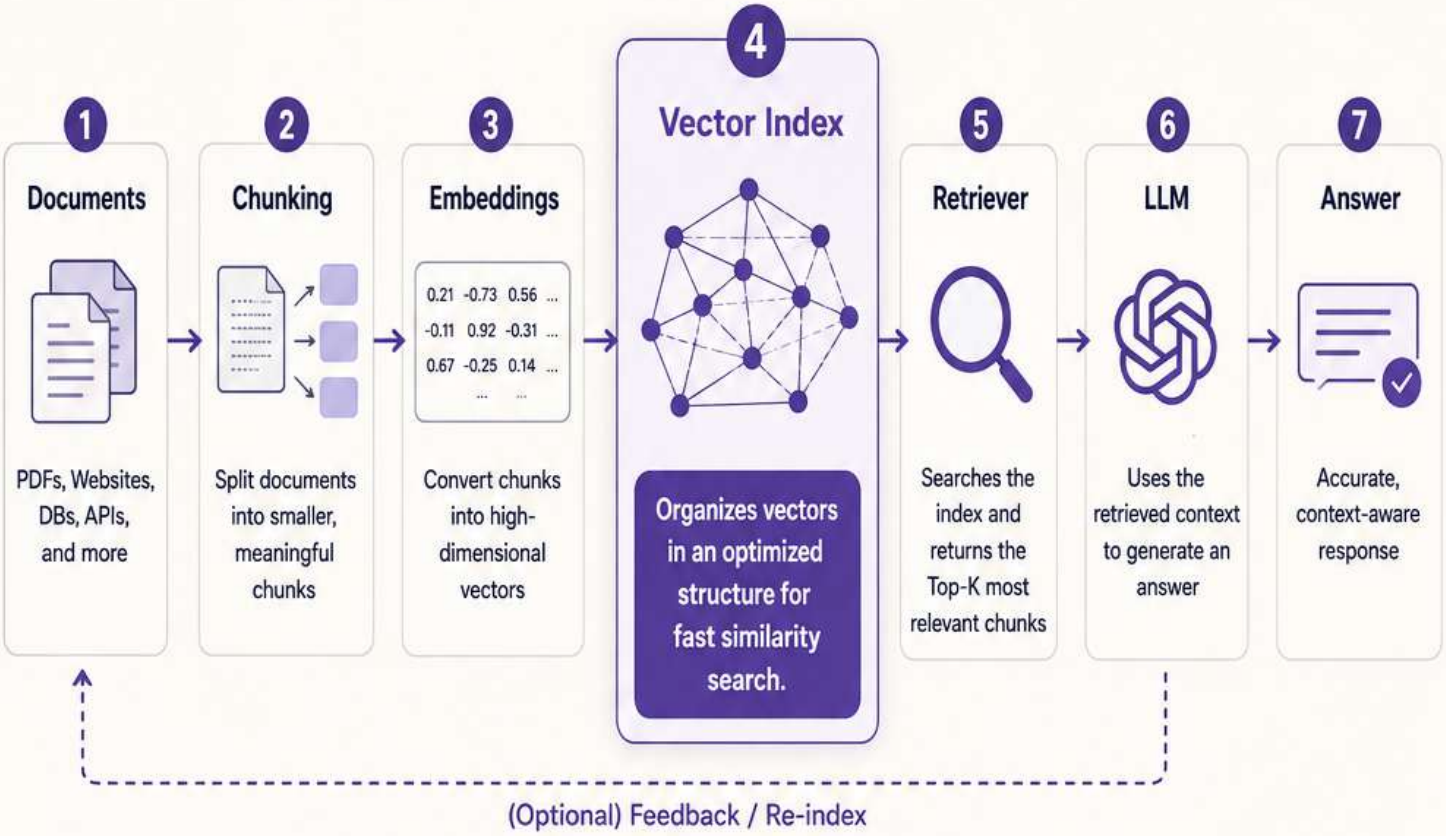
Key Takeaway

There is no one-size-fits-all index.
Choose the index based on your dataset size, latency needs, and accuracy requirements.

- Need highest accuracy? → Flat
- Need best overall balance? → HNSW
- Dataset is huge? → IVF or PQ
- Memory is limited? → PQ

Where the Index Fits in RAG

In a RAG system, the vector index is the engine that powers **fast and accurate** retrieval.



Why the Vector Index is Critical?

- Without an index → every query would scan the entire dataset (too slow).
- With an index → we search only promising candidates (milliseconds).
- Better index = faster retrieval + better answers.

Key Takeaway

The vector index sits at the heart of RAG.
It ensures the **right context** reaches the LLM—**fast.**
That's what makes RAG systems **powerful and scalable.**

Best Practices

Follow these practices to build **fast, accurate,** and **reliable** vector search systems.



1 Normalize Your Embeddings

- Normalize vectors (e.g., L2 normalization) before indexing.
- It improves similarity comparison and overall search accuracy.



2 Choose the Right Index

- Pick an index based on your dataset size, latency needs, and accuracy goals.
- There is no one-size-fits-all index.



3 Tune Top-K (and Thresholds)

- Experiment with different Top-K values.
- Use similarity score thresholds to filter low-relevance results.



4 Monitor Recall and Quality

- Regularly evaluate recall, precision, and context relevance.
- Use a validation set of queries to track retrieval quality over time.



5 Rebuild Indexes When Data Changes

- Additions, deletions, or major updates can degrade index quality.
- Rebuild or refresh indexes periodically for optimal performance.



6 Optimize for Your Environment

- Consider memory, storage, and compute trade-offs.
- Leverage hardware acceleration (e.g., GPU) when available.



Remember

Great retrieval is not just about the model—
it's about how well your **index is built and maintained.**

Challenges & Limitations

Vector indexes are powerful, but they come with trade-offs.
Knowing them helps you build **better systems.**

1 High Memory Usage

Indexes (especially graph-based ones) can consume a lot of memory.

Plan capacity carefully.

2 Curse of Dimensionality

As dimensions increase, distances become less meaningful.

Use the right embedding model & dimension size.

3 Approximate = Not Exact

ANN returns close neighbors, not the true nearest neighbors.

Balance accuracy and speed.

4 Index Staleness

Data changes can make the index outdated and less accurate.

Refresh or rebuild regularly.

5 Parameter Sensitivity

Performance depends on many parameters (Top-K, probes, efSearch, etc.).

Requires tuning and experimentation.

6 Cost at Scale

Large datasets mean higher compute, storage, and operational costs.

Optimize for your budget.

Trade-offs to Remember

What You Optimize For	You Gain	You Might Lose
Speed / Low Latency	Faster responses	Some accuracy
High Accuracy / Recall	More relevant results	Higher latency, more memory
Lower Memory Usage	Reduced infrastructure cost	Lower recall or slower search
Scalability	Handles more data	Higher complexity & tuning effort

Key Takeaway

There's no perfect index.
Understand the trade-offs and choose **what matters most** for your use case.